



Outline

Optimization

- Challenges in Optimization
- Momentum
- Adaptive Learning Rate
- **Parameter Initialization**
- Batch Normalization

Regularization of NN

- Norm Penalties
- Early Stopping
- Data Augmentation
- Sparse Representation
- Dropout



Parameter Initialization

- Goal: **break symmetry** between units
 - so that each unit computes a different function
- Initialize all weights (not biases) **randomly**
 - Gaussian or uniform distribution
- **Scale of initialization?**
 - *Large* -> grad explosion, *Small* -> grad vanishing



Xavier Initialization

- Heuristic for all outputs to have **unit variance**
- For a fully-connected layer with m inputs:

$$W_{ij} \sim N\left(0, \frac{1}{m}\right)$$

- For ReLU units, it is recommended:

$$W_{ij} \sim N\left(0, \frac{2}{m}\right)$$



Normalized Initialization

- Fully-connected layer with m inputs, n outputs:

$$W_{ij} \sim U\left(-\sqrt{\frac{6}{m+n}}, \sqrt{\frac{6}{m+n}}\right)$$

- Heuristic trades off between initialize all layers have same activation and gradient variance
- **Sparse** variant when m is large
 - Initialize k nonzero weights in each unit



Outline

Optimization

- Challenges in Optimization
- Momentum
- Adaptive Learning Rate
- Parameter Initialization
- Batch Normalization

Regularization of NN

- Norm Penalties
- Early Stopping
- Data Augmentation
- Sparse Representation
- Dropout



Feature Normalization

Good practice to normalize features before applying learning algorithm:

$$x' = \frac{x - \mu}{\sigma}$$

Feature vector $\rightarrow$ x Vector of mean feature values $\rightarrow$ μ
Vector of SD of feature values $\rightarrow$ σ

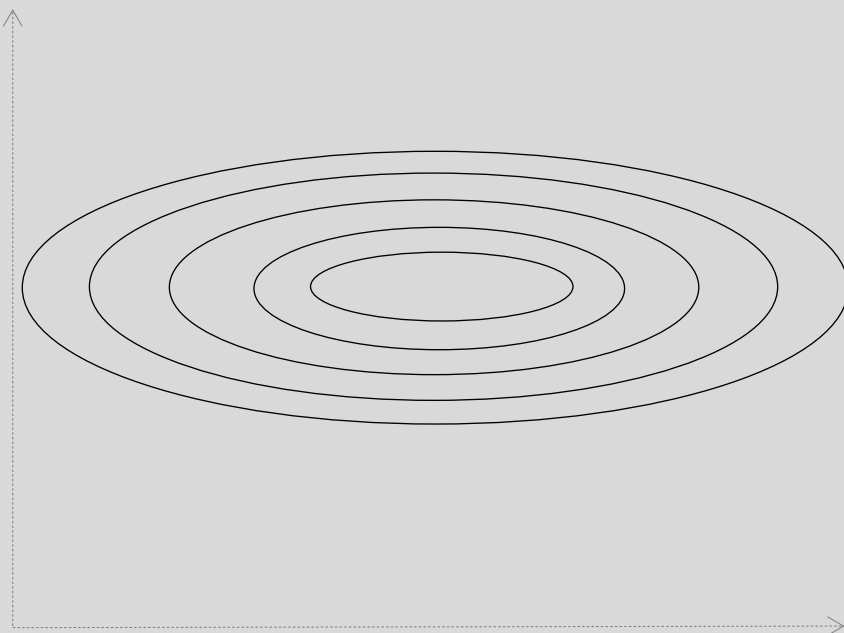
Features in **same scale**: mean 0 and variance 1

- Speeds up learning

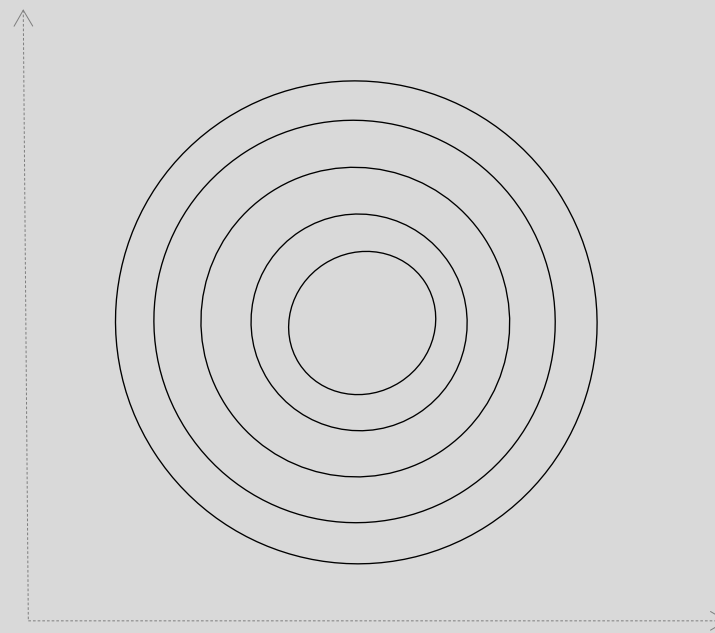


Feature Normalization

$L(W)$



Before normalization

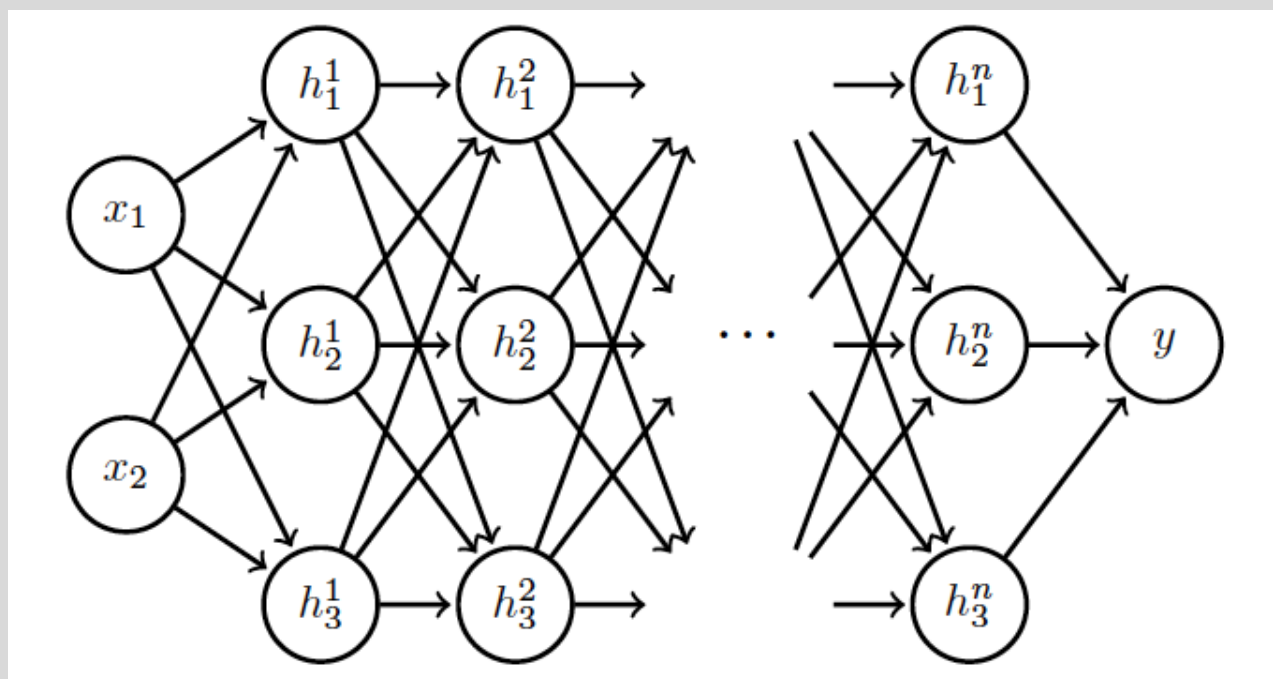


After normalization



Internal Covariance Shift

Each hidden layer changes distribution of inputs to next layer: *slows down learning*



Normalize
inputs to layer 2

...

Normalize
inputs to layer n



Batch Normalization

Training time:

- Mini-batch of activations for layer to normalize

$$H = \begin{bmatrix} \boxed{H_{11} \quad \cdots \quad H_{1K}} \\ \boxed{\vdots \quad \ddots \quad \vdots} \\ \boxed{H_{N1} \quad \cdots \quad H_{NK}} \end{bmatrix} \begin{matrix} K \text{ hidden layer} \\ \text{activations} \end{matrix}$$

N data points
in mini-batch



Batch Normalization

Training time:

- Mini-batch of activations for layer to normalize

$$H' = \frac{H - \mu}{\sigma}$$

where

$$\mu = \frac{1}{m} \sum_i H_{i,:}$$

Vector of mean activations
across mini-batch

$$\sigma = \sqrt{\frac{1}{m} \sum_i (H - \mu)_i^2 + \delta}$$

Vector of SD of each unit
across mini-batch



Batch Normalization

Training time:

- Normalization can reduce expressive power
- Instead use:

$$\gamma H' + \beta$$

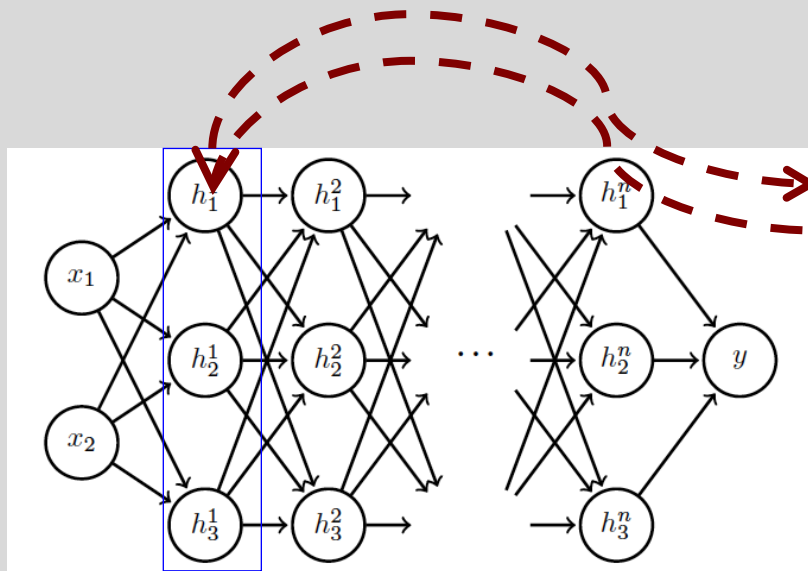
Learnable parameters

- Allows network to **control range of normalization**



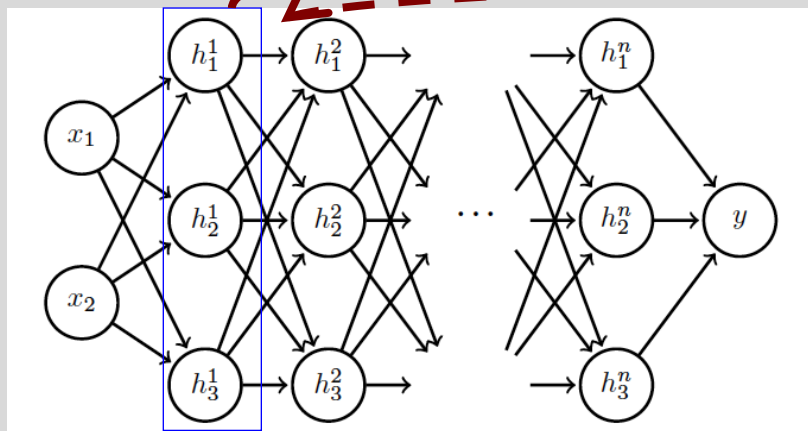
Batch Normalization

Batch 1



$$\mu^1 = \frac{1}{m} \sum_i H_{i,:}$$
$$\sigma^1 = \sqrt{\frac{1}{m} \sum_i (H - \mu)_i^2 + \delta}$$

Batch N

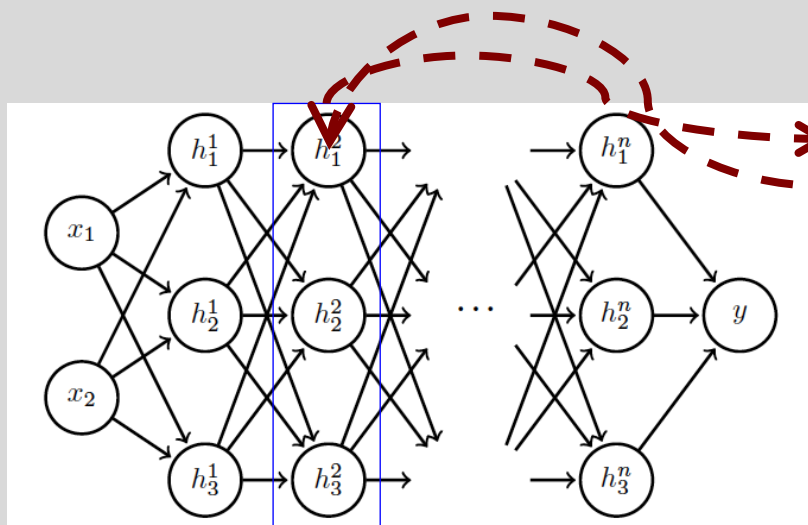


Add normalization
operations for
layer 1



Batch Normalization

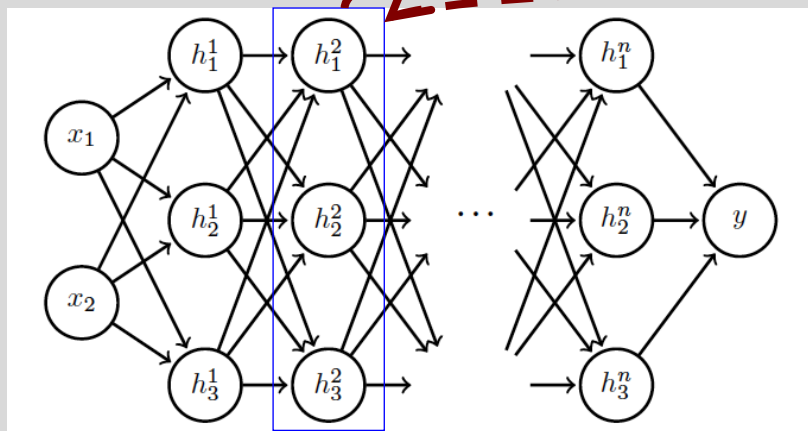
Batch 1



$$\mu^2 = \frac{1}{m} \sum_i H_{i,:}$$

$$\sigma^2 = \sqrt{\frac{1}{m} \sum_i (H - \mu)_i^2 + \delta}$$

Batch N



Add normalization operations for layer 2 and so on ...